

Assignment: Local Multi-LLM Coding Workflow Evaluation

Objective

Identify the most viable way to run a **local, multi-LLM workflow** that can collaboratively produce software artifacts (architecture, implementation, testing, documentation, deployment validation) using **local model backends**.

You must evaluate at least **two** candidate approaches and recommend one.

After your delivery: You must Review one of the other deliveries otherwise you risk not getting a pass on this assignment and will loose the oppotunity to go to exam in this class.

Hard Requirements

1. Multiple local endpoints

- Must support **minimum 2 separate local model endpoints** (e.g., two different local LLM servers/hosts).
- Must allow **switching** or **routing** between endpoints via configuration (not manual rewiring each time).

2. Open source

- Tooling must be **open source** (no requirement for OSI-approved license).

Functional Requirements

Your proposed solution must support a workflow that covers all responsibilities below. These can be implemented as distinct agents or as a structured pipeline, but **all must be covered**:

1. Architecture responsibility

- Produces architecture artifacts such as:
 - component decomposition and responsibilities
 - interface contracts (e.g., OpenAPI spec or equivalent)
 - deployment topology and constraints
 - architecture decision records (ADRs)

2. Tech lead responsibility

- Breaks work into **incremental tasks** (tickets) with:
 - scope boundaries
 - acceptance criteria (“definition of done”)
 - dependency ordering

3. Implementation responsibility

- Supports running **multiple coding workers** ($N \geq 2$) or a credible equivalent design that can parallelize/partition work.
- Produces multi-file changes across a repository.

4. Testing & quality responsibility

- Creates and runs tests (unit/integration as appropriate).
- Produces a quality report including:
 - test results
 - static checks (lint/type/etc. if relevant)
 - known limitations/risks

5. Documentation responsibility

- Produces developer and user documentation:
 - README (setup/run)
 - API usage (if relevant)
 - operational/runbook notes
 - Design documents

6. Deployment validation responsibility

- Validates deployability via at least one of:
 - a deployment checklist
 - a deployment script
 - container build/deploy config (if relevant)
 - environment/config documentation

Non-Functional Requirements

1. Predictability & control

- Must support at least one control mechanism:
 - ask-before-run (commands) and/or ask-before-edit (files), OR
 - plan + diffs for review before execution.

2. Reproducibility

- Must support version control workflows (git) and produce changes as commits or clearly reviewable diffs.
- Must be possible to run the same workflow twice and get **comparable** outputs (structure/progress), even if not identical.

3. Context management

- Must explain how the toolchain avoids **silent context loss** (artifact handoffs, summaries, scoping rules, etc.).
- Must explain how the workflow behaves when the project grows (repo size / docs size).

4. Security baseline

- Must not require exposing an unauthenticated model endpoint publicly as a prerequisite for the workflow.

Evaluation Requirements

Compare at least **two** candidate toolchains. For each, provide:

1. Setup complexity

- time to first working run
- moving parts (services, configs, dependencies)

2. Capability coverage

- what responsibilities are supported natively vs require custom glue

3. Multi-endpoint support

- how it connects to two separate local model endpoints
- how routing/switching is done
- whether different roles can be bound to different endpoints

4. Failure modes

- what breaks first (context, tool calling, file ops, test running)
- how failures are detected and recovered

5. Recommendation

- choose one approach and justify the decision against the requirements

Required Deliverables

1. synopsis

- 7-10 pages covering:
 - candidate toolchains compared
 - architecture of the recommended workflow
 - how it satisfies each functional requirement
 - tradeoffs, risks, and failure modes
 - recommendation and rationale
- *Review of another groups work.*

2. Short but complete setup guide

- Step-by-step instructions enabling a third party to reproduce:
 - installing prerequisites
 - configuring **two local model endpoints**
 - configuring the toolchain to use them
 - running a demo workflow that produces:
 - architecture output
 - at least one implemented feature
 - tests executed + results
 - docs updated
 - deployment validation step